

游戏 Square 大作业报告

孙广侑 2500094811 徐至晟 2400015881

一、程序功能介绍

用 C++ Qt 实现一个小游戏，玩家通过键盘方向键控制一个绿色方块，吃掉更小的方块以增大体型，碰到更大的方块游戏失败，吃掉所有敌人游戏胜利。

敌人方块是否可食用由边框颜色提示（绿色可吃，红色危险）。有不同种类的敌人方块，用不同颜色符号表示，以增加游戏难度和趣味性：沿直线运动的普通敌人、主动跟踪玩家的敌人、大小可伸缩的敌人、被吃会分裂的敌人等。

玩家吃掉一个敌人，有概率生成一个带问号的圆，吃掉圆（道具球）可以获得四种限时道具中的一种：屏障：免疫一次更大方块的碰撞；减速场：减速大部分敌人；磁铁：吸收周围可食用方块；缩小炸弹：缩小较大敌人 3 秒，但随后变大为原来的 160%。

玩家按空格可以加速自身，有冷却时间。按 Esc 暂停。

有四种难度可供选择：简单、中等、困难、地狱。难度越高，敌人方块数目越多，移动速度越快，大小越大。

界面包括游戏菜单、难度选择、规则页面、游戏主界面。游戏主界面上方文字显示难度、玩家方块大小、剩余敌人数、得分、道具是否激活等信息。胜利、失败、暂停会弹窗显示。

二、项目各模块与类设计细节

main.cpp: Qt 默认主程序

GameSquare.h 和 GameSquare.cpp: 玩家和敌人方块的类 GameSquare:

成员变量: XY 坐标、移动速度、边长、颜色。

成员函数:

GameSquare.cpp: 构造函数

GameSquare.h: 各个变量的 Get 和 Set 函数；移动函数 move（坐标加减）；类型转换为 QRect 的函数，方便使用 QRect 的 intersects 函数判断方块是否相交。

square.cpp: 游戏主体

Square 类继承自 QWidget，以下介绍成员变量和函数，按功能分类。

public:

```
explicit Square(QWidget *parent = nullptr);
```

```
~Square();
```

Square 类的构造函数和析构函数。

protected:

事件类:

```
void paintEvent(QPaintEvent *event) override;
```

绘制当前时刻游戏画面。

```
void keyPressEvent(QKeyEvent *event) override;
```

按下键盘，单独处理 Esc 键暂停和空格加速。

```
void keyReleaseEvent(QKeyEvent *event) override;
```

释放键盘按键。

private:

游戏状态初始化与更新:

```
void resetGameState();
```

重置游戏，玩家方块归位正中央，清空所有敌人、道具等信息

```
void startGame();
```

调用 resetGameState()和 initEnemies()，初始化敌人，重启计时器

```
void updateGame();
```

程序核心逻辑，更新游戏状态：不同类型敌人、道具球、玩家的运动及碰撞检测

```
void restartGame();
```

调用 startGame()

对话框：主菜单、难度选择、帮助、暂停、失败、胜利

```
void showMainMenu();
```

```
void showDifficultyDialog();
```

```
void showHelpDialog();
```

```
void showPauseDialog();
```

```
void showGGDialog();
```

```
void showVictoryDialog();
```

成员变量:

private:

```
enum class Difficulty {Easy, Medium, Tough, HellNo};
```

游戏难度编号：简单、中等、困难、地狱

```
enum class EnemyType {Normal, Hunter, Blinker, Splitter, Horror, Speedster};
```

敌人类型编号：普通、追踪（主动攻击）、闪烁（大小变化）、分裂、恐惧（被吃合并后的体型减小）、神速（无视减速场）

```
enum class PowerUpType {Shield, SlowField, Magnet, ShrinkBomb};
```

道具类型编号：屏障、减速场、磁铁、缩小炸弹

```
struct DifficultySettings {
```

```
    QString name;
```

```
    int enemyCount;
```

```
    int enemyBaseSize;
```

```
    int enemySizeStep;
```

```
    int enemySpeed;
```

```
    int playerSpeed;
```

```
    int scoreMultiplier;
```

```
    int starterEnemyCount;
```

```
    int maxPowerUps;
```

```
};
```

```
struct EatEffect {
```

```
    QPointF center;
```

```
    QColor color;
```

```
    int age;
```

```
    int maxAge;
```

```
    double startRadius;
```

```
};
```

视觉特效

```
struct PowerUp {
```

```
    QPointF center;
```

```

    PowerUpType type;

    int radius;

    int speedX;

    int speedY;

};

```

道具问号球的中心坐标、类型、半径、XY 移动速度
生成、修改、删除敌人的逻辑：

```

void initEnemies(); 初始化敌人

void setRandomDirection(GameSquare& enemy);

bool checkCollision(const GameSquare& enemy, int newX, int newY);

bool checkEnemyCollisions(int index, int newX, int newY);

bool isNearPlayer(const GameSquare& enemy, int margin) const;

void steerAwayFromPlayer(GameSquare& enemy);

void steerHunterTowardPlayer(int index, int step);

EnemyType randomEnemyType(int index) const;

难度越高，特殊敌人的出现概率越高

QColor enemyColorForType(EnemyType type) const;

void appendEnemy(const GameSquare& enemy, EnemyType type, int baseSize,
int phaseOffset);

void updateBlinkerEnemies();

void spawnSplitEnemies(const GameSquare& enemy, EnemyType sourceType);

int mergedSizeAfterEating(const GameSquare& enemy, EnemyType type) const;

void removeEnemyAt(int index);

```

道具：

```

void spawnPowerUp(const QPointF& center);

void setRandomPowerUpDirection(PowerUp& powerUp);

void movePowerUps(int speed);

void checkPowerUpCollisions();

void activatePowerUp(PowerUpType type);

void showPowerUpDialog(PowerUpType type);

```

```
QString powerUpName(PowerUpType type) const;
QString powerUpDescription(PowerUpType type) const;
void applyMagnetEffect();
void applyShrinkBomb();
void restoreShrinkBombEnemies();
void updateVisualEffects(); 视觉效果更新
void centerDialog(QDialog& dialog) const; 让弹窗在主窗口中心
    QString dialogStyleSheet() const; 弹窗统一样式表
```

难度设置:

```
void setDifficulty(Difficulty newDifficulty);
DifficultySettings currentSettings() const;
QString difficultyName() const;
```

```
QTimer *gameTimer;
QSet<int> pressedKeys;
Difficulty difficulty;
bool gameStarted;
```

储存玩家、敌人、道具信息、得分:

```
GameSquare player;
QList<GameSquare> enemies;
QList<EnemyType> enemyTypes;
QList<int> enemyBaseSizes;
QList<int> enemyPhaseOffsets;
QList<int> shrinkRestoreSizes;
QList<EatEffect> eatEffects;
QList<PowerUp> powerUps;
int score;
int powerUpsDropped;
```

边框:

```
int playerPulseFrames;
```

```
int dashFrames;  
  
int dashCooldownFrames;  
  
int shieldFrames;  
  
int slowFieldFrames;  
  
int magnetFrames;  
  
int shrinkBombFrames;  
  
int gameFrame;
```

三、小组成员分工情况

徐至晟：初步设计、初版游戏草稿、调试、大作业报告

孙广侑：功能升级、调试、游戏定稿、程序运行录屏

四、项目总结与反思

本次大作业，我们深度体验了游戏开发的全过程，从最开始的构思设计，到具备基本功能的初稿，再到功能升级、界面美观的最终稿。我们改进了生成方块的算法，修复了随机生成的方块重叠的问题；我们让敌人生成的位置与玩家保持一定距离，且保证有几个方块比玩家的初始大小更小，大大减少了“必输局”出现的情况，优化了玩家的游戏体验。

应用所学的 C++ 面向对象编程，并学习全新的 Qt，完成这个 1600 余行的大项目，大大锻炼了我们的自学能力、代码能力和实践应用能力。如何把庞大的项目化繁为简，一步步实现各项功能，团队协作至关重要。团队氛围融洽、沟通高效，有问题及时反馈，不断提出意见和新的创意。编程和调试的过程磨砺了我们耐心、细心、严谨的品格，这些都是未来学习和工作宝贵的财富。